HW2

University of Science and Industry of Iran Faculty of Computer Engineering

Computer Game Design Instructor: Dr. Behrooz Minaei Teaching Assistants: Amirhossein Mohammadi Pour and Mohammad Haghighat

Assignment 2

First Semester 1405-1406 (2026-2027)

Assignment Rules and Notes

- If you collaborate with other students or use resources outside the course content, you must cite the names of the individuals and the exact addresses of the sources.
- Name the project file as a compressed zip file `GD_Hw2_[firstName]_[lastName]_[StudentId]` and upload it to the course system.
- The responsibility for the integrity of the uploaded file lies with the student. Corrupted files will not receive a grade. Be sure to download and verify your file after uploading.

Practical Project: Implementation of an Infinite 3D Car Game

In this assignment, you must implement an **Endless 3D Car Game** using the **Unity** game engine. The player controls a car at night on an infinite road from a **Third-Person** perspective. The car is moving at high speed, and the player must steer between **3 lanes** to avoid collisions with obstacles, collect coins, and manage the car's fuel. In addition to strengthening technical skills in 3D development (procedural generation of the environment, optimization with Object Pooling), special attention should be given to visual effects (VFX), night lighting, and the artistic quality of the game.

Game Requirements and Features:

- **Environment and Procedural Road Generation:** The road must be generated procedurally. Road segments should be dynamically created ahead of the player

and recycled after the player passes them. The road consists of 3 lanes, and the player's car moves left/right between lanes using the **A/D** keys or arrow keys.

- **Input System:** All game inputs must be implemented exclusively using Unity's new **Input System** (Input Actions and Action Maps).
- **Important Note:** Projects implemented with the old Input system will not receive points in this section.

Gameplay and Car Control:

- The player's car has **3 Lives**, the number of which is displayed to the player in the UI.
- Colliding with any obstacle car deducts one life and must be accompanied by appropriate visual feedback (such as the car changing color or camera shake).
- When the number of lives reaches zero, the game ends (**Game Over**).

Fuel System:

- The player's car has a fuel tank that decreases over time.
- **Fuel Cans** are placed along the path; collecting them recharges the car's fuel.
- Running out of fuel is equivalent to game over, and the remaining fuel must be displayed as a fuel bar in the UI.
- **Progressive Difficulty:** With the passage of time and increasing distance traveled, the game difficulty should increase by gradually increasing the car's speed and the density of obstacle cars on the road.
- **Optimization with Object Pooling:** All repeating objects in the game (road segments, obstacle cars, coins, and fuel cans) must be managed using the **Object Pooling** pattern. Creating and destroying duplicate objects with `Instantiate`/`Destroy` is not acceptable.
- **OOP Design:** The game must include at least two types of obstacles (static and moving cars) and two types of collectible items (coins and fuel cans). To adhere to object-oriented principles, all these elements must use an abstract class named `Interactable` and override its virtual methods.
- **Singleton Pattern for Game Management:** Game state management (Menu, Playing, Paused, GameOver) must be done through a `GameManager` class using the Singleton design pattern.
- **Visual Effects, Lighting & VFX (Visual Art, Lighting & VFX):** In addition to the technical logic of the game, the artistic aspects and visual appeal of the game

constitute an important part of this assignment's evaluation, and implementing the following items carries bonus points:

- **Night Environment:** The game environment must be set at night. Use an appropriate night **Skybox** (or a procedural sky), local environment lighting, and the necessary night lighting for the road.
- **Dynamic Neon Lighting:** The player's car must have a neon light under its body (**Underglow Neon**) in addition to the main headlights. The **Intensity** and brightness of the neon light should be adjusted proportionally to the car's speed (the faster the speed, the brighter and more radiant the neon light should be). Additionally, the presence of streetlights along the road path is recommended.
- **Sky Fireworks:** For the beauty and visual appeal of the night sky, a fireworks system must be implemented in the sky such that fireworks are launched (randomly) from any point and create a beautiful colorful glow in the sky.
- **Particle System & Post-Processing:** Using the **Particle System** for car exhaust smoke/fire, collision sparks, and item fade-out effects is essential. Also, using the **Bloom** effect (from the Post-Processing package) is mandatory for the realistic glow of neon lights and fireworks explosions.
- **Audio System:** The game must include appropriate sound design: background music, car engine sound, collision sound with obstacles, and item collection sound (coins and fuel).
- **User Interface (UI):**
- **Main Menu:** Includes start game and exit buttons.
- **Pause Menu:** Includes resume, restart, and return to main menu buttons.
- **Game Over Screen:** Displays the current score, highest score record, and restart button.
- **In-Game:** Live display of car speed, fuel bar, remaining lives, current score, and highest score record.
- **Data Persistence:** Using the `PlayerPrefs` system, the highest score record (High Score) must be saved so that it is preserved after exiting the game.

Submission Instructions:

You must place your Unity project in a Git repository (such as GitHub or GitLab). The final file submitted to the system must be a compressed zip file containing the following:

1. A text file containing the project's Git repository link (make sure to use a `.gitignore` file so that bulky folders like the `Library` are not uploaded to Git, and only the essential files for running the project are included).
2. The executable **Build** file of the game.
3. A short work report including an explanation of the implementation structure and images of the final game output.